

Тестовое задание: AI-оператор / AI-агент (CityCar)

Дворцов Михаил Борисович

1. Архитектура решения

Запрос руководителя попадает к агенту-оркестратору, который разбирает его и запускает цепочку. Расчёты ведёт код, а текст отчёта пишет модель только по готовым цифрам.

Агенты и компоненты:

1. Оркестратор. Понимает запрос и период, строит план, собирает отчёт.
2. Сборщик из amoCRM. Через API v4 берёт сделки, задачи, статусы, менеджеров (/leads, /tasks, /users) с учётом лимита 7 запросов в секунду. Снимок за 30 дней пишет в PostgreSQL.
3. Сборщик телефонии. Через API провайдера получает список звонков за период и ссылки на записи.
4. Обработчик звонков. Записи скачиваются, расшифровываются в текст (STT, например Whisper или SpeechKit) с разделением по голосам, затем модель по каждому звонку возвращает структурированный JSON: тема, возражения, исход, следующий шаг, соответствие скрипту. Расшифровки и JSON хранятся в PostgreSQL, аудио в объектном хранилище с ограниченным сроком.
5. Аналитик. Считает метрики кодом (конверсия по этапам, скорость ответа, сделки без задач и с просроченными, число звонков и длительность, разрез по менеджерам), без участия модели.
6. Автор отчёта. Модель пишет текст только по этим цифрам и цитатам из звонков.
7. Проверяющий. Сверяет числа в тексте с расчётами и отмечает расхождения.

Планирование: очередь задач (Celery и Redis), звонки обрабатываются фоном, готовый отчёт приходит в мессенджер или на почту. Результаты и история отчётов хранятся в PostgreSQL.

2. Что делает система и что передаёт человеку

Сама: собирает данные, считает метрики, расшифровывает и разбирает звонки, находит сделки без задач и с просроченными, пишет отчёт.

Человеку: любые выводы о конкретном менеджере с последствиями (премия, увольнение), спорные звонки с низкой уверенностью разбора, отчёты, где данные CRM явно неполные, решение, что менять в работе отдела.

Три главных риска:

1. Качество данных. Менеджеры могут не вести CRM, и отчёт будет красивым, но неверным. Система должна показывать долю заполненных данных и предупреждать.
2. Персональные данные и записи звонков (152-ФЗ). Нужны уведомление о записи, ограничение срока хранения и доступ только руководителю.
3. Ошибки и галлюцинации модели: неверная расшифровка, придуманные цифры, несправедливая оценка менеджера. Поэтому цифры считает код, модель только формулирует, а выводы о людях подтверждает человек.

3. Код: сделки без задач или с просроченными задачами

Файл: amo_deals_check.py (Python, около 40 строк). Логика поиска проблемных сделок проверена на тестовых данных, к реальной amoCRM код не подключался: нужно подставить поддомен и токен.

```
"""Сделки amoCRM без задач или с просроченными задачами (тестовое задание CityCar).
```

```
Подставьте SUBDOMAIN и ACCESS_TOKEN. Код не запускался против реальной amoCRM:
логика find_problem_deals проверена на тестовых данных.
```

```
"""
```

```
import time
```

```
import requests
```

```
BASE = "https://SUBDOMAIN.amocrm.ru/api/v4"
```

```
HEADERS = {"Authorization": "Bearer ACCESS_TOKEN"}
```

```
CLOSED = {142, 143} # системные статусы amoCRM: успешно / не реализовано
```

```
def fetch_all(path, key, params=None):
```

```
    """Постранично читает список из amoCRM. Ответ 204 = данных больше нет."""
```

```
    page, items = 1, []
```

```
    while True:
```

```
        r = requests.get(f"{BASE}/{path}", headers=HEADERS, timeout=30,
                        params=**((params or {}), "page": page, "limit": 250))
```

```
        if r.status_code == 204:
```

```
            return items
```

```
        r.raise_for_status()
```

```
        items += r.json()[ "_embedded" ][key]
```

```
        page += 1
```

```
        time.sleep(0.15) # лимит amoCRM: 7 запросов в секунду
```

```
def find_problem_deals(deals, tasks, now):
```

```
    by_deal = {}
```

```
    for t in tasks:
```

```
        by_deal.setdefault(t["entity_id"], []).append(t)
```

```
    result = []
```

```
    for d in deals:
```

```
        open_tasks = by_deal.get(d["id"], [])
```

```
        if not open_tasks:
```

```
            result.append({"deal_id": d["id"], "problem": "no_tasks"})
```

```
        elif any(t["complete_till"] < now for t in open_tasks):
```

```
            result.append({"deal_id": d["id"], "problem": "overdue_task"})
```

```
    return result
```

```
def deals_without_or_overdue_tasks():
```

```
    deals = [d for d in fetch_all("leads", "leads") if d["status_id"] not in CLOSED]
```

```
    tasks = fetch_all("tasks", "tasks",
```

```
                    {"filter[entity_type]": "leads", "filter[is_completed]": 0})
```

```
    return find_problem_deals(deals, tasks, int(time.time()))
```

```
if __name__ == "__main__":
```

```
    for item in deals_without_or_overdue_tasks():
```

```
        print(item)
```

4. Реальный AI-проект

RAG-ассистент поддержки для сотрудников компании (команда из двух человек, на реальных данных клиента: 104 000 тикетов).

- Задача: сотрудники ждали оператора часами (среднее закрытие тикета 47 часов), хотя решения были в архиве.
- стек: Python, FastAPI, ChromaDB, эмбединги multilingual-e5, YandexGPT, MS SQL, Telegram, Docker.
- Что делает: ищет похожие решения, отвечает, при неуверенности передаёт вопрос оператору с карточкой похожих решений, обновляет индекс раз в сутки, считает аналитику на дашборде.

- Моя часть: набор проверок качества и подбор порогов эскалации, оценки ответов и калибровка весов, классификация обращений, авторизация API, настройка контейнеров и автообновление.
- Результат: измеренных результатов в эксплуатации нет. Цель была ответ за секунды вместо часов.

5. Чему научился за последние полгода и где применил

Спец-driven разработка с ИИ-агентами и защита инфраструктуры. До кода пишу спецификацию и критерии приёмки, ставлю агентам задачи с ограничениями (какие каталоги трогать, что запрещено), проверяю результат тестами и линтерами. Применил на платформе с персональными данными: по результатам двух аудитов безопасности усилил защиту контейнеров (запрет повышения привилегий, ограничения ресурсов, файловая система только для чтения), закрепил версию действия в CI и добавил автотест на ограничение запросов.

6. Улучшение, которое предложил и довёл до результата

В боте для клуба я заметил, что статистика засчитывала подписку тому, кто быстрее нажал кнопку, а не тому, кто реально подписался на канал. Это искажало стоимость подписчика по объявлениям. Я перевёл учёт на факт подтверждённой подписки, закрепил метку источника за человеком навсегда и добавил автотесты на атрибуцию. После этого воронка и стоимость подписчика считаются по реальным подписчикам.